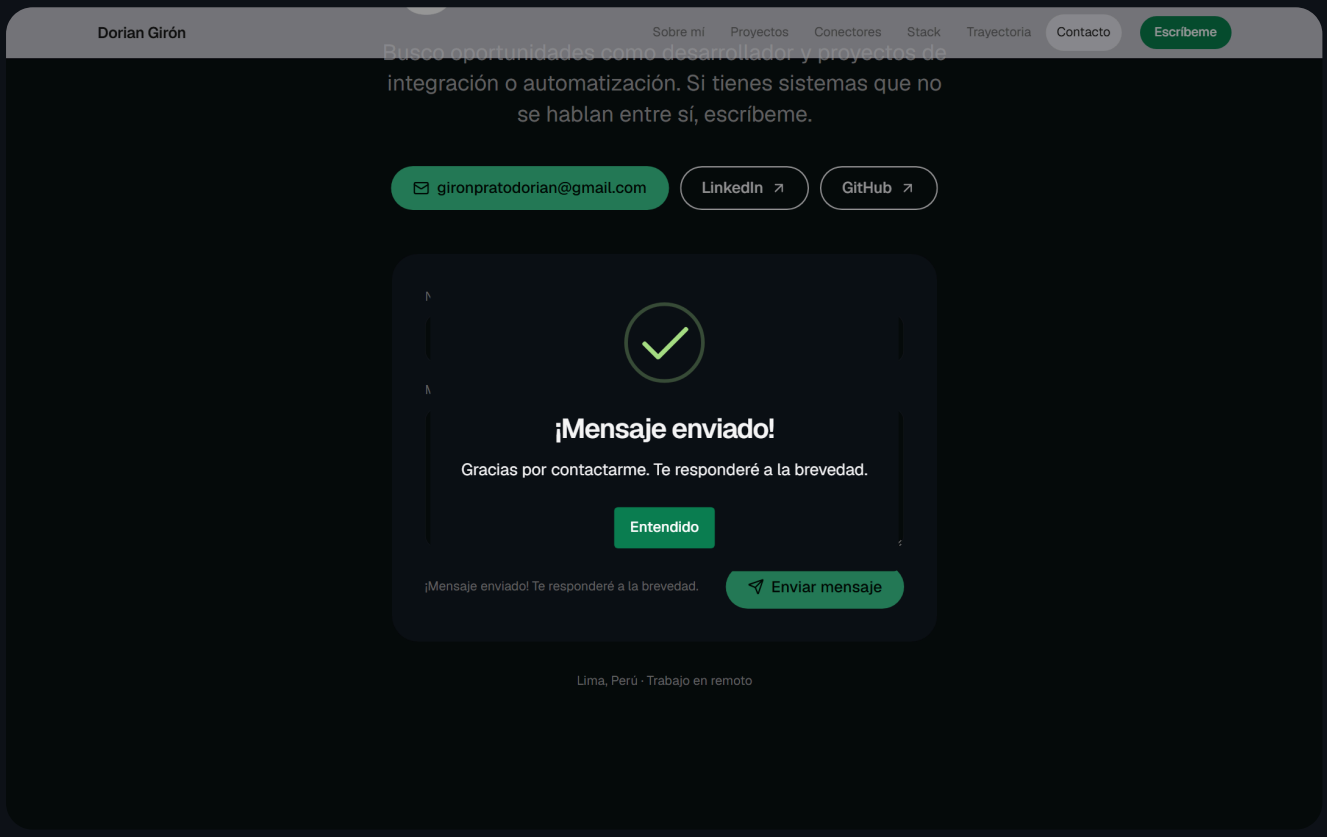
SENATI

Escuela de Tecnologías de la Información

Laboratorio #6 · Guía Práctica 6

Formulario funcional con EmailJS



Curso	Fullstack Developer Software – Seminario (IA 501)
Instructor	Ing. Giancarlos Barboza N.
Alumno	Dorian Girón Prato
Carrera	Ingeniería de Software con Inteligencia Artificial · 5.º ciclo
Sitio publicado	dorian-portafolio.vercel.app
Repositorio	github.com/Dgirto/dorian-portfolio
Fecha	25 de octubre de 2026

Documentación con capturas de configuración, código y ejecución.

Contenido

Índice

1. Objetivo, tecnologías y funcionamiento
2. Paso 1 · Configuración en el panel de EmailJS
3. Paso 2 · Variables de entorno en Vite
4. Paso 3 · Instalación de librerías
5. Paso 4 · Componente del formulario conectado a EmailJS
6. Publicación en Vercel con las variables de entorno
7. Ejecución: envío real, estado de carga, éxito y error
8. Verificación de calidad
9. Seguridad, diferencias con la guía y conclusiones

1. Objetivo

Convertir el formulario de contacto del portafolio, que en el Laboratorio #4 era un formulario controlado sin servidor, en un **formulario funcional**: al pulsar «Enviar mensaje», el mensaje llega por correo electrónico al propietario del sitio. Para lograrlo sin construir un backend propio se usa **EmailJS**, un servicio que envía correos desde el navegador a partir de una plantilla, y **SweetAlert2** para mostrar el resultado con una alerta.

Tecnologías

Herramienta	Uso
EmailJS (`@emailjs/browser` 5)	Envía el correo con un servicio (Gmail) y una plantilla, sin servidor propio.
SweetAlert2	Alerta de éxito o error al terminar el envío.
Vite (`import.meta.env`)	Lee las credenciales desde el archivo `.env` (variables con prefijo `VITE_`).
React 19 + TypeScript	Formulario controlado con `useState` y envío asíncrono con `async/await` .
Vercel	Publicación del sitio y variables de entorno de producción.

Cómo funciona

1. Visitante
Completa nombre, correo y mensaje en el formulario del portafolio.

→

2. React
`emailjs.send()` envía los datos con el Service ID, el Template ID y la Public Key.

→

3. EmailJS
Completa la plantilla (`{{nombre}}` , `{{email}}` , `{{mensaje}}`) y envía el correo por Gmail.

→

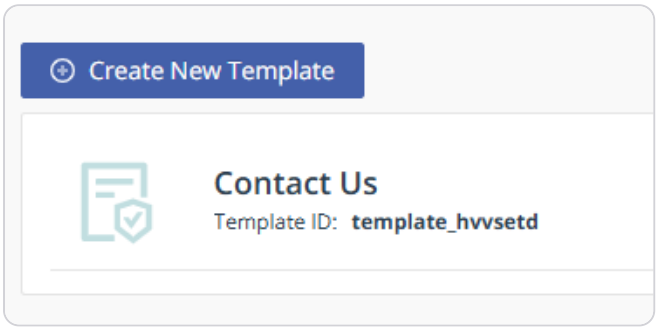
4. Correo
Llega a gironpratodorian@gmail.com con «Responder a» apuntando al visitante.

Paso 1

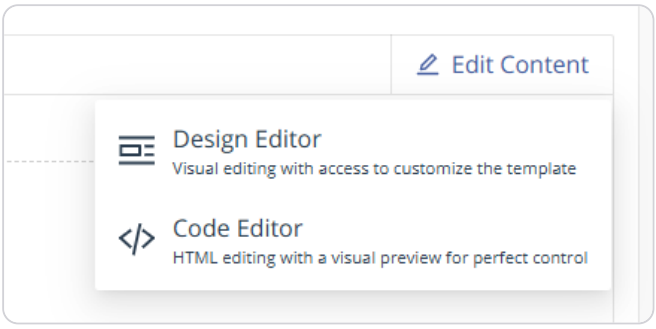
Configuración en el panel de EmailJS

Antes de escribir código se crearon las credenciales en emailjs.com con una cuenta gratuita (200 envíos al mes).

1. **Email Services → Add New Service:** se conectó la cuenta de Gmail. EmailJS asignó el **Service ID** `service_xa2vuyz`.
2. **Email Templates → Create New Template:** se creó la plantilla «Contact Us», con Template ID `template_hvvsetd`.
3. **Account → API Keys:** se copió la **Public Key** (`kHZV...`), que identifica la cuenta al enviar desde el navegador.



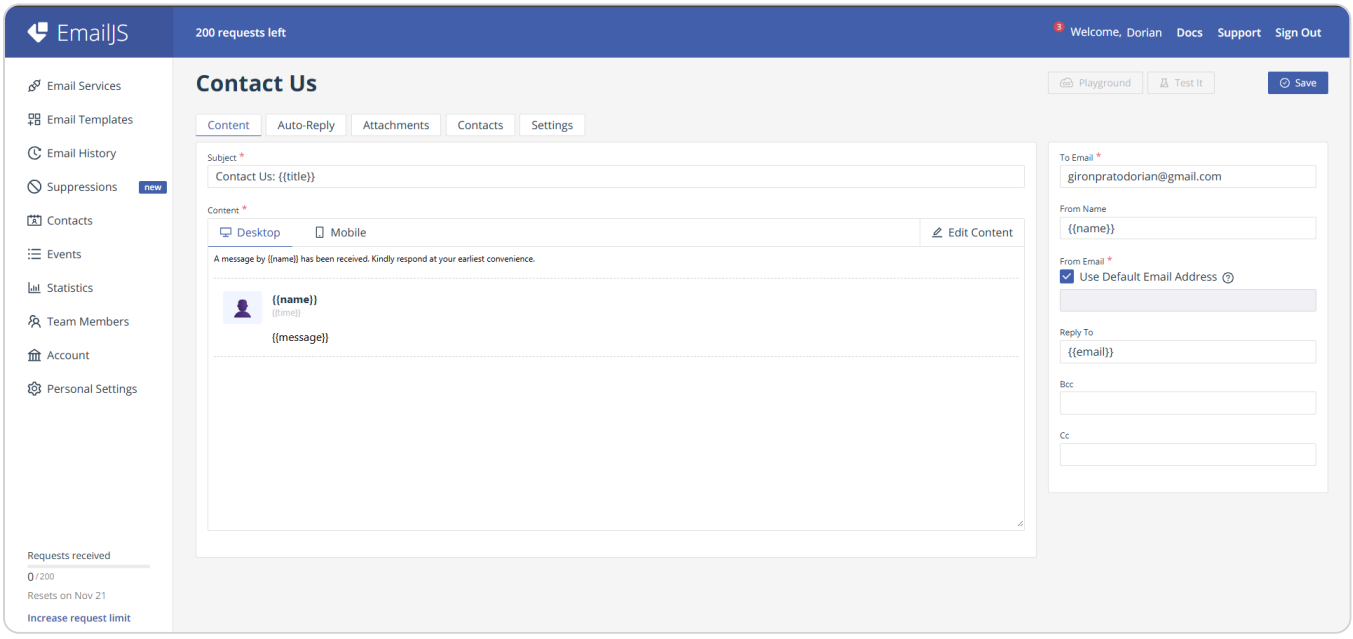
Plantilla creada y su Template ID.



«Edit Content»: editor visual o editor de código HTML.

La plantilla

En la plantilla se definen el destinatario (**To Email**), el asunto y el cuerpo. Las variables entre llaves dobles se reemplazan por los valores que envía React. Se configuró **Reply To** = `{{email}}` para responder al visitante con un clic.



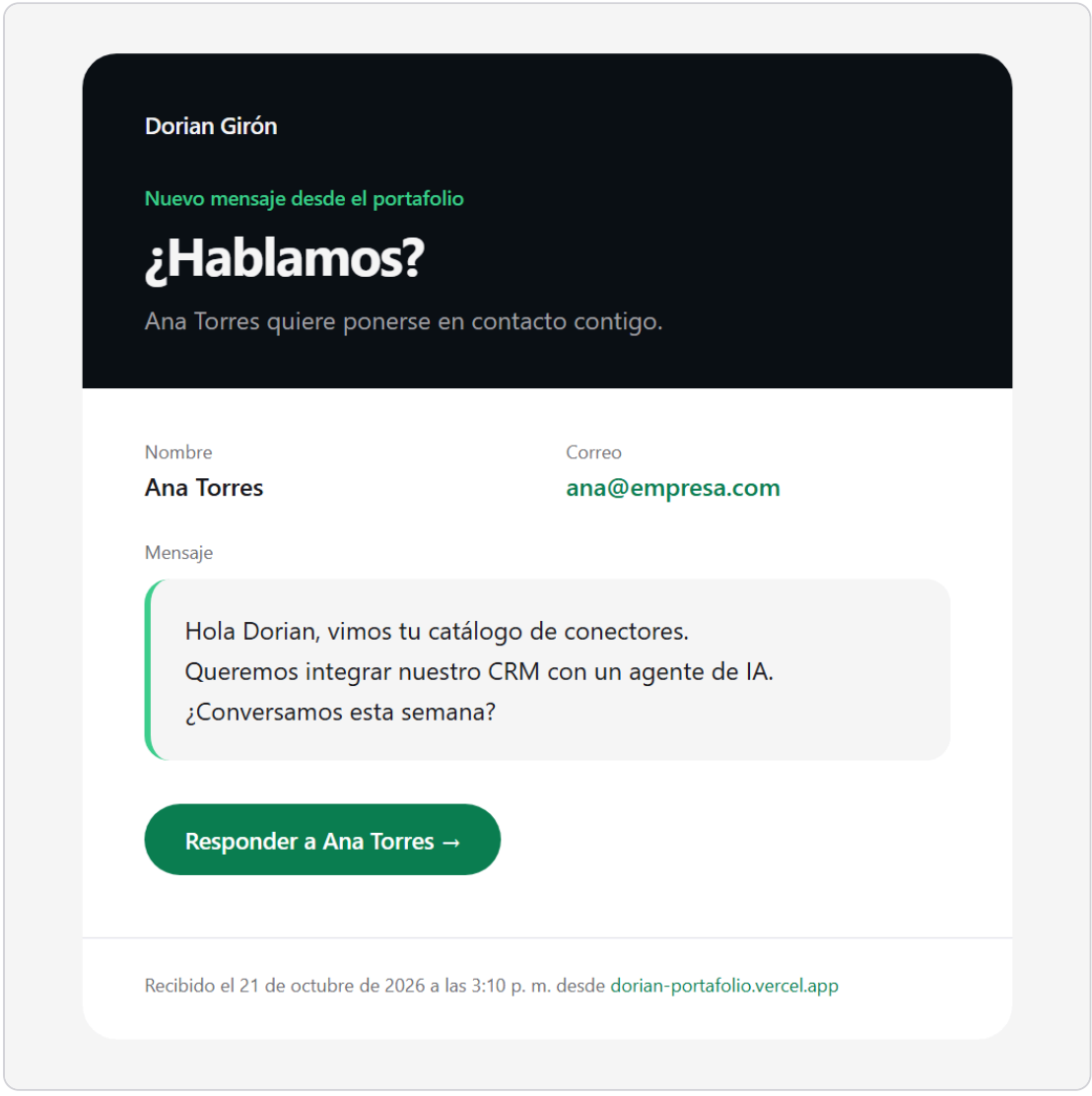
Configuración de la plantilla en EmailJS: asunto, contenido, To Email, From Name y Reply To.

Variable	Origen en React	Uso en la plantilla
`{{nombre}}` / `{{name}}`	`form.nombre`	Remitente y saludo
`{{email}}`	`form.email`	Reply To y datos del visitante
`{{mensaje}}` / `{{message}}`	`form.mensaje`	Cuerpo del correo
`{{title}}` , `{{time}}`	Generados al enviar	Asunto y fecha de recepción

> La guía pide `{{nombre}}` , `{{email}}` y `{{mensaje}}` . La plantilla «Contact Us» que crea EmailJS por defecto usa `{{name}}` , `{{message}}` , `{{title}}` y `{{time}}` , así que el formulario envía **ambos juegos de nombres**: funciona con cualquiera de las dos plantillas.

Diseño del correo

Para que el correo recibido tenga la misma identidad que el portafolio se diseñó una plantilla HTML (Code Editor de EmailJS) con tablas y estilos en línea, que es lo que soportan Gmail y Outlook.



Diseño de la plantilla con datos de ejemplo: cabecera oscura «¿Hablamos?», datos del visitante, mensaje y botón para responder.

Plantilla HTML de EmailJS (Code Editor) · líneas 1–30

```html
<div style="margin:0; padding:32px 12px; background:#f5f5f7; font-family:-apple-system, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;">
  <table role="presentation" width="100%" cellpadding="0" cellspacing="0" border="0" style="max-width:600px; margin:0 auto; border-collapse:separate;">

    <!-- Cabecera oscura, como los capítulos del portafolio -->
    <tr>
      <td style="background:#0d1117; border-radius:22px 22px 0 0; padding:36px 40px 32px;">
        <p style="margin:0 0 28px; font-size:15px; font-weight:600; letter-spacing:-0.01em; color:#f5f5f7;">Dorian Girón</p>
        <p style="margin:0 0 10px; font-size:13px; font-weight:600; color:#3ecf8e;">Nuevo mensaje desde el portafolio</p>
        <h1 style="margin:0; font-size:34px; line-height:1.1; font-weight:700; letter-spacing:-0.03em; color:#f5f5f7;">¿Hablamos?</h1>
        <p style="margin:12px 0 0; font-size:16px; line-height:1.5; color:#a1a1a6;">{{nombre}} quiere ponerse en contacto contigo.</p>
      </td>
    </tr>

    <!-- Cuerpo claro con la ficha del visitante y el mensaje -->
    <tr>
      <td style="background:#ffffff; padding:32px 40px 8px;">
        <table role="presentation" width="100%" cellpadding="0" cellspacing="0" border="0">
          <tr>
            <td width="50%" style="padding:0 12px 20px 0; vertical-align:top;">
              <p style="margin:0 0 4px; font-size:12px; color:#6e6e73;">Nombre</p>
              <p style="margin:0; font-size:16px; font-weight:600; color:#1d1d1f;">{{nombre}}</p>
            </td>
            <td width="50%" style="padding:0 0 20px 12px; vertical-align:top;">
              <p style="margin:0 0 4px; font-size:12px; color:#6e6e73;">Correo</p>
              <p style="margin:0; font-size:16px; font-weight:600;"><a href="mailto:{{email}}" style="color:#0a7f52; text-decoration:none;">{{email}}</a></p>
            </td>
          </tr>
        </table>

        <p style="margin:4px 0 10px; font-size:12px; color:#6e6e73;">Mensaje</p>
```

Paso 2

Variables de entorno en Vite

Las credenciales no se escriben en el código fuente. En la raíz del proyecto (junto a `package.json`) se creó el archivo `.env` con las tres variables. En Vite, solo las variables que empiezan por `VITE_` llegan al navegador.

El archivo real `.env` no se sube a GitHub; en el repositorio queda `.env.example` como modelo:

.env.example

```
# Copia este archivo como .env y completa los valores de tu panel de EmailJS (emailjs.com).
# Vite solo expone al navegador las variables que empiezan por VITE_. Reinicia `npm run dev` tras editarlo.
# La plantilla de EmailJS debe usar las variables {{nombre}}, {{email}} y {{mensaje}}.
VITE_EMAILJS_SERVICE_ID=service_xxxxxxx
VITE_EMAILJS_TEMPLATE_ID=template_xxxxxxx
VITE_EMAILJS_PUBLIC_KEY=xxxxxxxxxxxxxxxx
```

Tras crear o editar `.env` hay que **reiniciar** `npm run dev`: Vite solo lee el archivo al arrancar.

> **Punto clave 1 de la guía:** en Vite las variables se leen con `import.meta.env.VITE_…`, no con `env()` como en Laravel ni con `process.env` como en Node o Webpack. Además, Vite las **incrusta al compilar**: si se cambian, hay que volver a compilar y desplegar.

Exclusión del repositorio

.gitignore · líneas 7–10

```
# env & secrets
.env
.env.*
!.env.example
```

Terminal · dorian-portfolio

```
> git check-ignore -v .env
.gitignore:8:.env       .env
```

Tipos de las variables (TypeScript)

Como el proyecto usa TypeScript, se declararon las variables para que `import.meta.env` tenga autocompletado y comprobación de tipos:

src/vite-env.d.ts

```ts
/// <reference types="vite/client" />

// Credenciales públicas de EmailJS (ver .env.example). Si faltan, el formulario abre el correo del visitante.
interface ImportMetaEnv {
  readonly VITE_EMAILJS_SERVICE_ID?: string
  readonly VITE_EMAILJS_TEMPLATE_ID?: string
  readonly VITE_EMAILJS_PUBLIC_KEY?: string
}

interface ImportMeta {
  readonly env: ImportMetaEnv
}
```

Paso 3

Instalación de librerías

Se instalaron EmailJS y SweetAlert2 desde la terminal del proyecto:

Terminal · dorian-portfolio

```
> npm install @emailjs/browser sweetalert2
```

Versiones instaladas, comprobadas al generar este documento:

Terminal · dorian-portfolio

```
> npm ls @emailjs/browser sweetalert2
dorian-portfolio@1.0.0 E:\proyecto portafolio
+-- @emailjs/browser@5.0.3
`-- sweetalert2@11.26.25
```

package.json · dependencias · líneas 14–24

```
14    },
15    "dependencies": {
16      "@emailjs/browser": "^5.0.3",
17      "@fontsource-variable/geist": "^5.3.0",
18      "lucide-react": "^1.59.0",
19      "motion": "^14.3.0",
20      "react": "^19.3.1",
21      "react-dom": "^19.3.1",
22      "sweetalert2": "^11.26.25",
23      "three": "^0.186.1"
24    },
```

Carga bajo demanda

Las dos librerías se importan con `import()` dinámico dentro del envío. Así no pesan en la carga inicial de la página: Vite las separa en archivos propios que el navegador solo descarga cuando alguien envía el formulario.

Terminal · dorian-portfolio

```
> npm run build
dist/assets/sweetalert2.all-CkxkDIIb.js                  78.01 kB │ gzip:  20.81 kB
dist/assets/index-B7GdpXV2.js                           427.89 kB │ gzip: 137.47 kB
✓ built in 778ms
```

Paso 4

Componente del formulario conectado a EmailJS

La guía actualiza `Contacto.jsx`. En este proyecto el formulario vive en `src/components/ContactForm.tsx` (el mismo componente controlado del Laboratorio #4) y se conectó a EmailJS con programación asíncrona.

Guía (Contacto.jsx)	Este proyecto (ContactForm.tsx)
`formulario` / `setFormulario`	`form` / `setForm` (un único estado con los tres campos)
`cargando` (booleano)	`status` : `idle · sending · sent · error` ; `sending = status === 'sending'`
`manejarCambio`	`onChange` : actualiza el campo según su atributo `name`
`manejarEnvio` con `async/await`	`onSubmit` con `async/await` y `try/catch`
`Swal.fire(...)`	`alerta(ok)` : una sola función para éxito y error
`disabled={cargando}`	`disabled={sending}` en los tres campos y el botón

Credenciales desde el .env

src/components/ContactForm.tsx · líneas 15–32

```
15  // Credenciales públicas de EmailJS, leídas del .env (prefijo VITE_). No son secretas: EmailJS limita su uso
16  // a los dominios permitidos en su panel.
17  const EMAILJS = {
18    serviceId: import.meta.env.VITE_EMAILJS_SERVICE_ID,
19    templateId: import.meta.env.VITE_EMAILJS_TEMPLATE_ID,
20    publicKey: import.meta.env.VITE_EMAILJS_PUBLIC_KEY,
21  }
22  const emailjsReady = Boolean(EMAILJS.serviceId && EMAILJS.templateId && EMAILJS.publicKey)
23
24  const NOTES: Record<Status, string> = {
25    idle: emailjsReady
26      ? 'Tu mensaje me llega directamente al correo.'
27      : 'Al enviar se abrirá tu aplicación de correo con el mensaje ya redactado.',
28    sending: 'Enviando tu mensaje…',
29    sent: '¡Mensaje enviado! Te responderé a la brevedad.',
30    error: 'No se pudo enviar. Inténtalo de nuevo o escríbeme directamente al correo.',
31    opened: 'Se abrió tu aplicación de correo con el mensaje listo. Solo falta pulsar Enviar.',
32  }
```

Si falta alguna variable, el formulario no finge un envío: abre la aplicación de correo del visitante con el mensaje redactado (comportamiento del Laboratorio #4). Los textos de `NOTES` se anuncian a los lectores de pantalla con `aria-live`.

Estado controlado y manejo de cambios

src/components/ContactForm.tsx · líneas 57–66

```tsx
export function ContactForm({ to }: { to: string }) {
  const [form, setForm] = useState<Fields>(EMPTY)
  const [status, setStatus] = useState<Status>('idle')
  const id = useId()
  const sending = status === 'sending'

  const onChange = (e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) => {
    const { name, value } = e.target
    setForm((prev) => ({ ...prev, [name]: value }))
  }
```

Envío asíncrono (manejarEnvio)

src/components/ContactForm.tsx · líneas 75–105

```tsx
  const onSubmit = async (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault()
    if (!emailjsReady) return openMailClient()

    setStatus('sending')
    try {
      const { default: emailjs } = await import('@emailjs/browser')
      // Variables de la guía ({{nombre}}, {{email}}, {{mensaje}}) y también las de la plantilla
      // «Contact Us» por defecto de EmailJS ({{name}}, {{message}}, {{title}}, {{time}}).
      const params = {
        ...form,
        name: form.nombre,
        message: form.mensaje,
        title: `Mensaje de ${form.nombre} desde el portafolio`,
        time: new Date().toLocaleString('es-PE', { dateStyle: 'long', timeStyle: 'short' }),
      }
      await emailjs.send(EMAILJS.serviceId!, EMAILJS.templateId!, params, {
        publicKey: EMAILJS.publicKey,
        // Antispam: un envío cada 10 s por visitante y sin navegadores automatizados.
        limitRate: { id: 'contacto', throttle: 10_000 },
        blockHeadless: true,
      })
      setForm(EMPTY)
      setStatus('sent')
      await alerta(true)
    } catch (error) {
      console.error('Error al enviar el mensaje:', error)
      setStatus('error')
      await alerta(false)
    }
  }
```

- `e.preventDefault()` evita que el navegador recargue la página.
- `setStatus('sending')` activa el estado de carga antes de llamar a EmailJS.
- `await emailjs.send(serviceId, templateId, params, opciones)` espera la respuesta del servicio.
- Si todo va bien, se limpia el formulario y se muestra la alerta de éxito; si falla, el error queda en la consola y se muestra la alerta de error **sin borrar lo escrito**, para que el visitante pueda reintentar.
- `limitRate` permite un envío cada 10 segundos y `blockHeadless` rechaza navegadores automatizados (antispam).

Alertas con SweetAlert2

src/components/ContactForm.tsx · líneas 34–50

```tsx
/** Alerta con SweetAlert2, cargada solo al enviar para no pesar en la carga inicial. */
async function alerta(ok: boolean) {
  const { default: Swal } = await import('sweetalert2')
  await Swal.fire({
    title: ok ? '¡Mensaje enviado!' : 'No se pudo enviar',
    text: ok
      ? 'Gracias por contactarme. Te responderé a la brevedad.'
      : 'Hubo un problema al enviar el mensaje. Inténtalo más tarde o escríbeme directamente al correo.',
    icon: ok ? 'success' : 'error',
    confirmButtonText: 'Entendido',
    confirmButtonColor: '#0a7f52',
    background: '#0d1117',
    color: '#f5f5f7',
    // Sin esto, SweetAlert2 fija la altura del body y la página salta al inicio mientras la alerta está abierta.
    heightAuto: false,
  })
}
```

Los colores siguen la paleta del portafolio (fondo `#0d1117`, botón verde `#0a7f52`). `heightAuto: false` evita que la página salte al inicio mientras la alerta está abierta.

Campos y botón bloqueados durante el envío

src/components/ContactForm.tsx · líneas 107–162

```tsx
  return (
    <form className={styles.form} onSubmit={onSubmit} aria-describedby={`${id}-note`} aria-busy={sending}>
      <div className={styles.row}>
        <div className={styles.field}>
          <label htmlFor={`${id}-nombre`}>Nombre</label>
          <input
            id={`${id}-nombre`}
            name="nombre"
            type="text"
            autoComplete="name"
            value={form.nombre}
            onChange={onChange}
            placeholder="Tu nombre"
            required
            disabled={sending}
          />
        </div>
        <div className={styles.field}>
          <label htmlFor={`${id}-email`}>Correo</label>
          <input
            id={`${id}-email`}
            name="email"
            type="email"
            autoComplete="email"
            value={form.email}
            onChange={onChange}
            placeholder="tu@correo.com"
            required
            disabled={sending}
          />
        </div>
      </div>
      <div className={styles.field}>
```

```
        <label htmlFor={`${id}-mensaje`}>Mensaje</label>
        <textarea
          id={`${id}-mensaje`}
          name="mensaje"
          rows={5}
          value={form.mensaje}
          onChange={onChange}
          placeholder="Cuéntame sobre tu proyecto o la oportunidad…"
          required
          disabled={sending}
        />
      </div>
      <div className={styles.footer}>
        <p id={`${id}-note`} className={styles.note} aria-live="polite">
          {NOTES[status]}
        </p>
        <button type="submit" className="btn" disabled={sending}>
          <Send size={18} aria-hidden="true" /> {sending ? 'Enviando…' : emailjsReady ? 'Enviar mensaje' :
'Preparar correo'}
        </button>
      </div>
    </form>
  )
}
```

Punto clave 2 de la guía: con `disabled={sending}` los campos y el botón se bloquean mientras se envía, así un usuario impaciente no puede enviar el mismo correo varias veces. `aria-busy` lo comunica a los lectores de pantalla.

Punto clave 3 de la guía (`finally`): la guía usa `finally` para devolver `cargando` a `false` . Aquí el estado de carga no es un booleano aparte, sino que se deriva de `status` : tanto el `try` (`'sent'`) como el `catch` (`'error'`) lo sacan de `'sending'` , por lo que el botón siempre vuelve a su estado normal, igual que con `finally` .

Uso en la sección de contacto

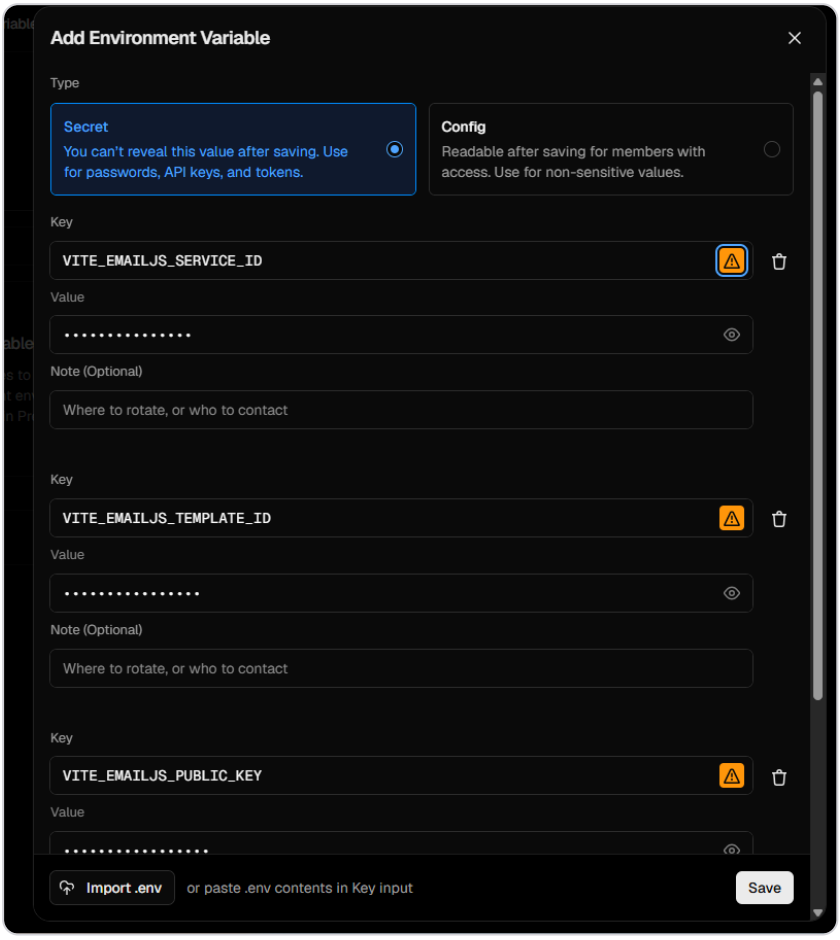
src/sections/Contact.tsx · líneas 38–40

```
        <Reveal delay={0.25}>
          <ContactForm to={profile.email} />
        </Reveal>
```

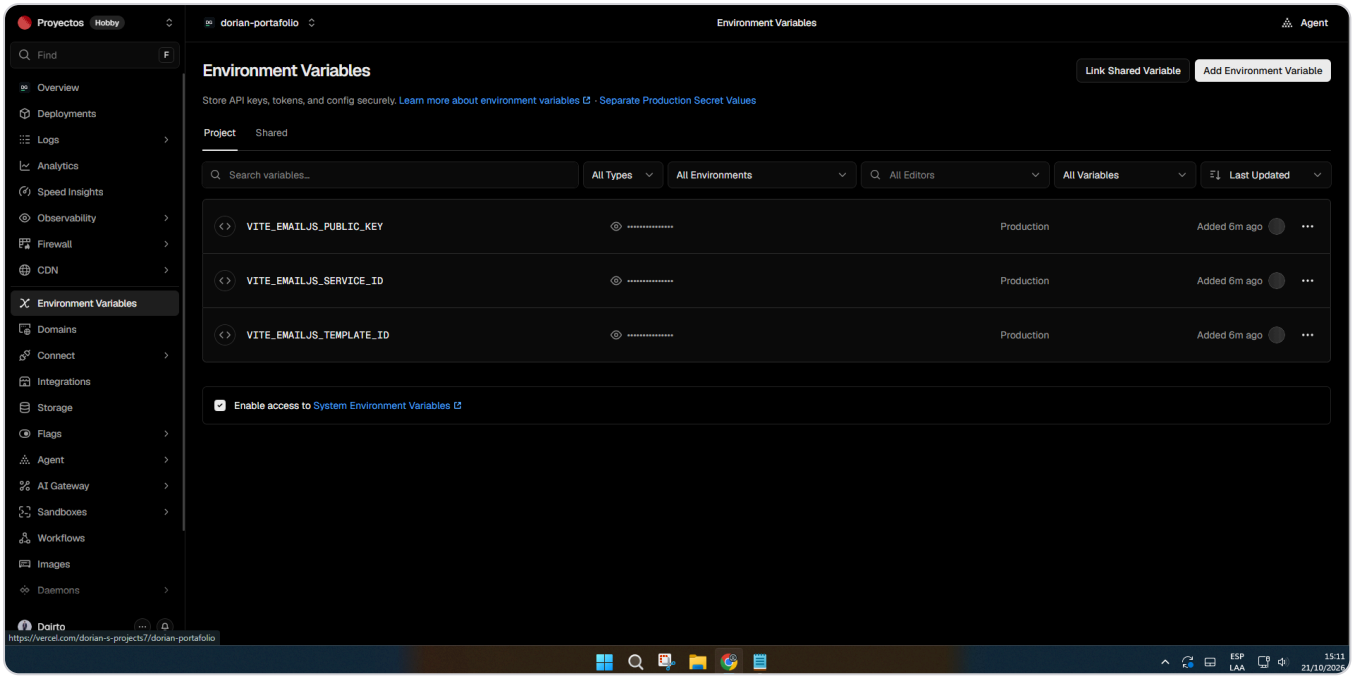
Publicación

Variables de entorno en Vercel

El archivo `.env` solo existe en la computadora de desarrollo. Para que el sitio publicado envíe correos, las mismas tres variables se añadieron en Vercel: **Project** → **Settings** → **Environment Variables**, entorno **Production**.



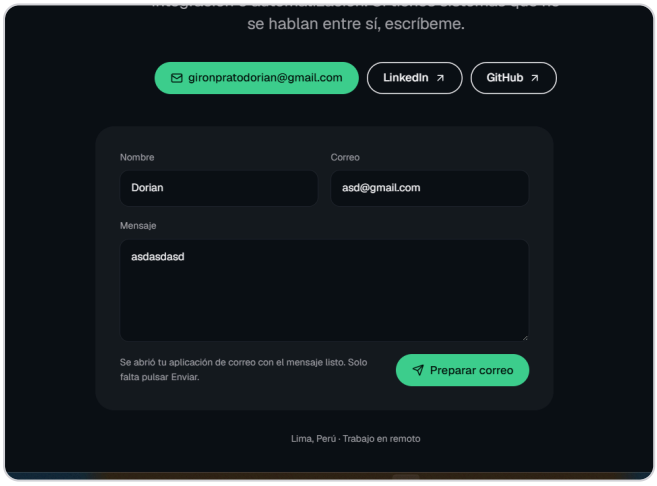
Alta de las tres variables en Vercel. El aviso naranja indica que una variable VITE_ queda visible en el navegador: es lo esperado para la Public Key de EmailJS.



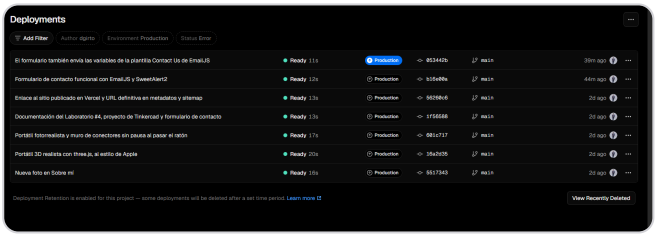
Variables guardadas en el proyecto dorian-portafolio, entorno Production.

Por qué hizo falta volver a desplegar

Al guardar las variables, el sitio seguía mostrando «Preparar correo». Vite incrusta `import.meta.env` al compilar, así que el despliegue anterior se había compilado **sin** las claves. Se resolvió con **Deployments** → ⋯ → **Redeploy**.



Antes del Redeploy: sin variables, el formulario usa el respaldo (abrir el correo del visitante).



Despliegues del proyecto en Vercel (cada push a main publica una versión nueva).

Comprobación del sitio publicado

Este script descarga el JavaScript publicado y confirma que las credenciales quedaron incrustadas:

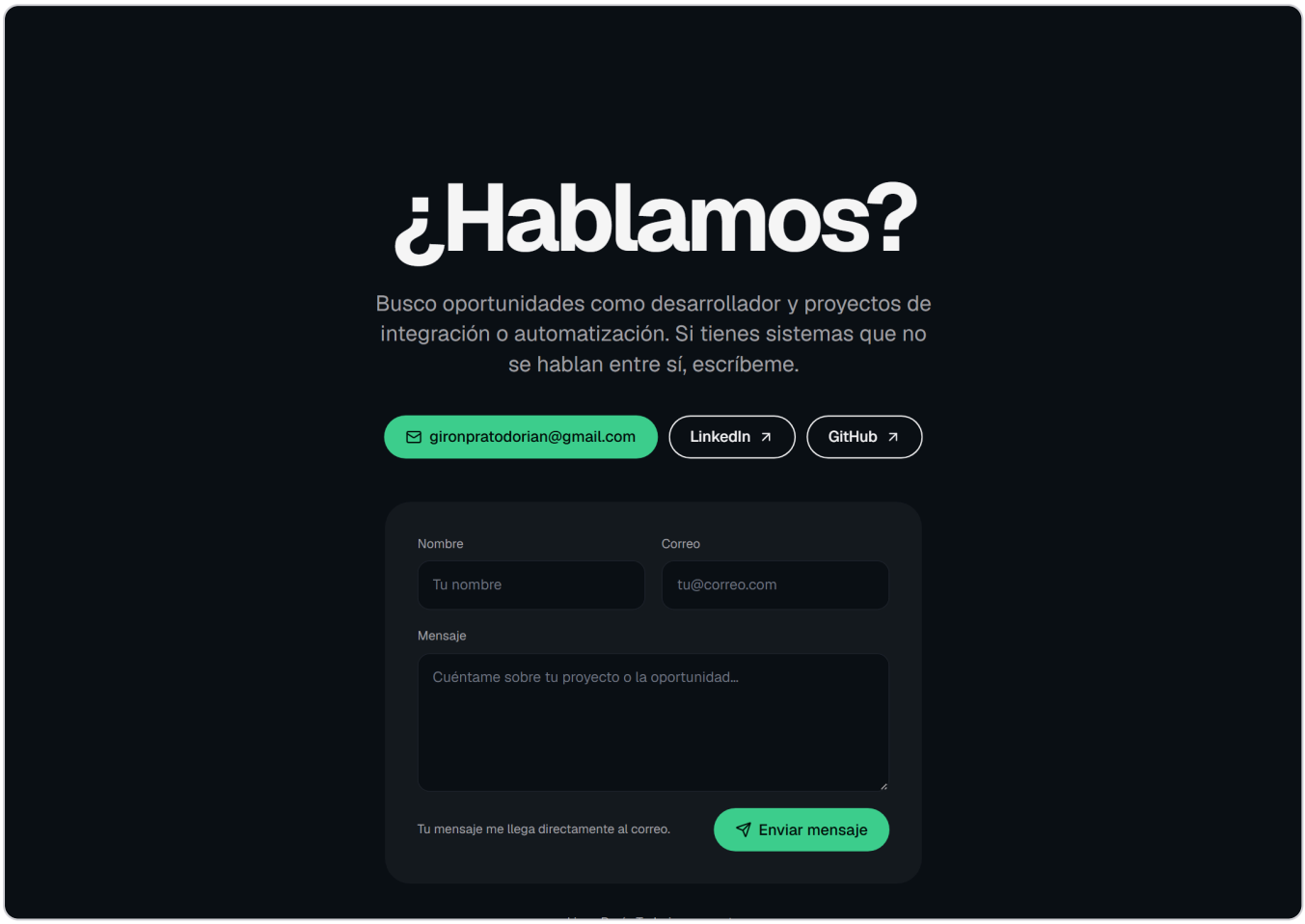
Terminal · dorian-portfolio

```
> node docs/herramientas/verificar-produccion.mjs
Sitio: https://dorian-portafolio.vercel.app/
Bundle principal: assets/index-CqR1e2Lh.js
Service ID incrustado: service_xa2vuyz
Template ID incrustado: template_hvvsetd
Resultado: el formulario envía con EmailJS.
```

Ejecución

Prueba en el sitio publicado

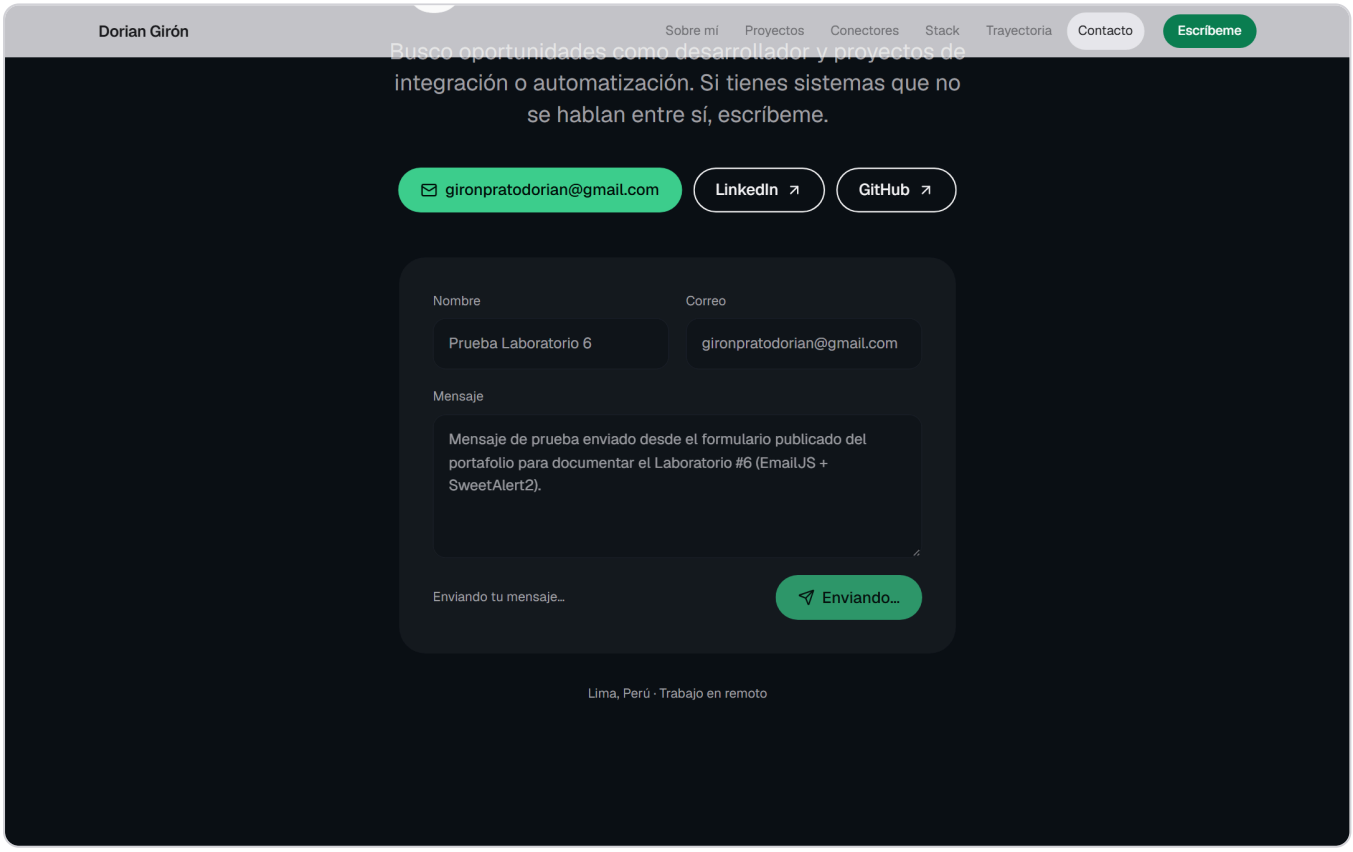
Prueba realizada en dorian-portafolio.vercel.app con un **envío real** el 25 de octubre de 2026. Para que el estado «Enviando…» se viera en la captura, se simuló una red lenta en el navegador de prueba.



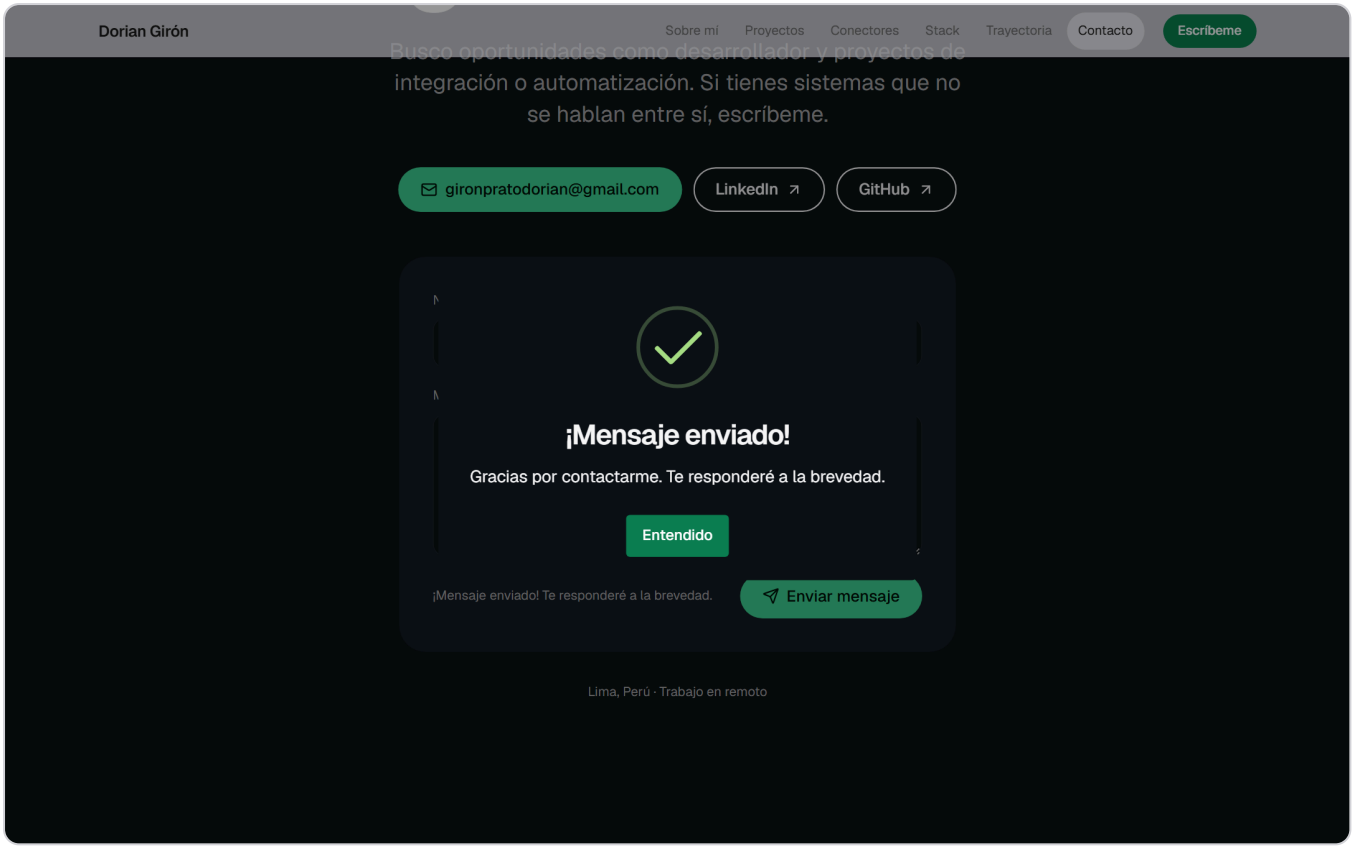
1. Sección de contacto con el formulario listo: la nota indica «Tu mensaje me llega directamente al correo».



2. Formulario completado con nombre, correo y mensaje.



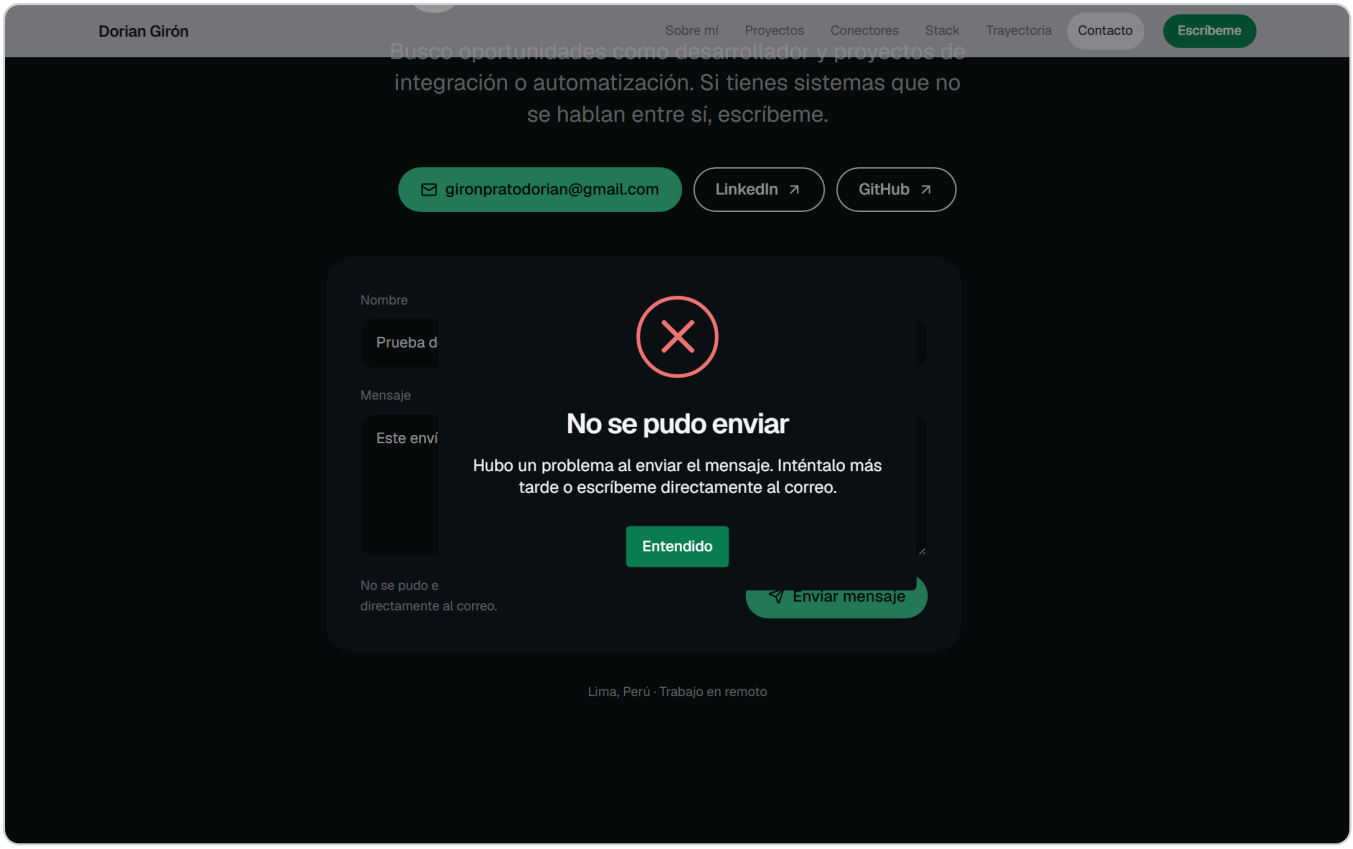
3. Durante el envío: botón «Enviando...», campos y botón deshabilitados y nota «Enviando tu mensaje...».



4. Respuesta real de EmailJS: alerta «¡Mensaje enviado!» de SweetAlert2 y formulario vacío.

Manejo de errores

Para documentar el caso de error se **simuló una pérdida de conexión** en el navegador de prueba (no se envió nada). La petición a EmailJS falla, se muestra la alerta de error y el texto escrito se conserva para reintentar.



5. Sin conexión: alerta «No se pudo enviar»; los datos del formulario se mantienen.

Calidad

Verificación

Salidas reales de los comandos del proyecto, ejecutados al generar este documento.

Terminal · dorian-portfolio

```
> npm run lint
(sin salida: sin errores)
```

Terminal · dorian-portfolio

```
> npm run typecheck
(sin salida: sin errores)
```

Terminal · dorian-portfolio

```
> npm test


 Test Files  4 passed (4)
      Tests  15 passed (15)
   Start at  23:16:22
   Duration  439ms (import 39%, transform 25%, tests 19%, worker 15%, environment 1%)
```

Terminal · dorian-portfolio

```
> npm run build
dist/assets/geist-latin-wght-normal-BgDaEnEv.woff2         29.40 kB
dist/assets/index-De8m1dAm.css                             38.62 kB │ gzip:   9.34 kB
dist/assets/esm-C2ePPWuE.js                                 3.48 kB │ gzip:   1.48 kB
dist/assets/sweetalert2.all-CkxkDIIb.js                    78.01 kB │ gzip:  20.81 kB
dist/assets/index-B7GdpXV2.js                             427.89 kB │ gzip: 137.47 kB
dist/assets/laptopScene-D0i9qtKk.js                       549.54 kB │ gzip: 137.56 kB

✓ built in 908ms
```

Control de versiones

Terminal · dorian-portfolio

```
> git log --oneline -8
3dfc02a Nuevo proyecto: Tuerka, sitio de venta de vehículos llevado de Figma a React
053442b El formulario también envía las variables de la plantilla Contact Us de EmailJS
b16e00a Formulario de contacto funcional con EmailJS y SweetAlert2
56260c6 Enlace al sitio publicado en Vercel y URL definitiva en metadatos y sitemap
1f56588 Documentación del Laboratorio #4, proyecto de Tinkercad y formulario de contacto
601c717 Portátil fotorrealista y muro de conectores sin pausa al pasar el ratón
16a2d35 Portátil 3D realista con three.js, al estilo de Apple
5517343 Nueva foto en Sobre mí
```

Cierre

Seguridad, diferencias con la guía y conclusiones

Seguridad

- **La Public Key de EmailJS es pública por diseño:** viaja en el JavaScript del sitio. Lo que impide que otros la usen es la lista de dominios permitidos en **Account** → **Security** (solo `dorian-portafolio.vercel.app` y `localhost`).
- `.env` está en `.gitignore`: las credenciales no están en el historial de GitHub.
- Antispam: un envío cada 10 segundos por visitante (`limitRate`) y bloqueo de navegadores automatizados (`blockHeadless`).
- Campos `required` y `type="email"`: el navegador valida antes de enviar.

Diferencias con la guía

Guía	Este proyecto	Motivo
`Contacto.jsx` (JavaScript)	`ContactForm.tsx` (TypeScript)	Tipos para el estado y las variables de entorno.
Booleano `cargando` + `finally`	Estado `status` con cuatro valores	Un solo estado sirve para el botón, la nota accesible y las alertas.
Imports estáticos	`import()` dinámico	EmailJS y SweetAlert2 no pesan en la carga inicial.
Variables `{{nombre}}`, `{{email}}`, `{{mensaje}}`	Esas y las de la plantilla por defecto	Funciona con la plantilla de la guía y con «Contact Us».
Sin respaldo	Si faltan credenciales, abre el correo del visitante	El formulario nunca finge un envío que no ocurre.
Solo en local	Publicado en Vercel con variables de producción	El formulario funciona en el sitio real.

Conclusiones

- EmailJS permite que un sitio estático envíe correos reales sin programar ni mantener un servidor.
- Separar las credenciales en variables de entorno evita exponerlas en el repositorio y permite usar valores distintos en local y en producción.
- En Vite las variables se incrustan al compilar: tras cambiarlas en Vercel hay que volver a desplegar.
- `async/await` con `try/catch` y el bloqueo del formulario durante el envío dan una experiencia clara y evitan correos duplicados.

> Sitio: https://dorian-portafolio.vercel.app · Código: https://github.com/Dgirto/dorian-portfolio